

Smart Student Expense & Budget Analyzer

1. Overview

The **Smart Student Expense & Budget Analyzer** is a beginner-friendly Python command-line application designed to help students manage their personal finances.

The application allows students to record and manage expenses, set budgets, track income, manage financial goals, analyze spending, and generate a final financial report.

The project is developed using Python programming concepts covered in **CSE1021 - Introduction to Problem Solving and Programming**.

2. Features

Expense Management

- Add expenses.
- View all expenses.
- Search expenses.
- Filter expenses by category.
- Edit expenses.
- Delete expenses.

Budget Management

- Set a monthly budget.
- Set category-wise budgets.
- View budget status.
- Check budget warnings.

Expense Analytics

- Calculate total spending.
- Calculate average expense.
- Find the highest expense.
- Find the lowest expense.
- Calculate category-wise spending.
- Count transactions.

Financial Goals

- Add financial goals.
- View financial goals.
- Add savings to goals.
- Delete goals.
- Track goal progress.

Income Management

- Set income.
- View current income.

Final Report

- View income.
- View total spending.
- View remaining money.
- View average expense.
- View highest and lowest expenses.
- View category-wise spending.
- View budget status.
- View financial goals.

3. Technologies Used

- **Programming Language:** Python
- **Interface:** Command-Line Interface (CLI)
- **Data Storage:** JSON
- **Testing:** Python `unittest`
- **Version Control:** Git and GitHub

No external Python libraries are required.

4. Project Structure

```
Smart_Student_Expense_Budget_Analyzer/
```

```
|
|  └─ main.py
|  └─ README.md
|  └─ statement.md
|
|  └─ data/
|      └─ budget.json
|      └─ expenses.json
|      └─ goals.json
|      └─ income.json
|
|  └─ modules/
|      └─ budget_manager.py
|      └─ expense_manager.py
|      └─ analytics.py
|      └─ goal_manager.py
|      └─ income_manager.py
|      └─ report_generator.py
|      └─ __init__.py
|
|  └─ utils/
|      └─ file_handler.py
|      └─ validation.py
|      └─ __init__.py
|
|  └─ tests/
|      └─ test_budget.py
|      └─ test_expenses.py
|      └─ test_analytics.py
|      └─ test_goals.py
|      └─ test_income.py
|      └─ test_report.py
|      └─ __init__.py
```

5. Requirements

Before running the project, make sure Python 3.x is installed.

Check the Python version using:

```
```bash
python --version
```

The project does not require any external Python packages.

## 6. Installation and Setup

---

### Step 1: Download the Project

Download or clone the project from GitHub:

```
git clone https://github.com/Shabd26/Smart-Student-Expense-and-Budget-Analyzer.git
```

### Step 2: Open the Project Folder

Move into the project directory:

```
cd Smart-Student-Expense-and-Budget-Analyzer
```

Make sure the folder contains:

```
main.py
modules/
utils/
data/
tests/
README.md
statement.md
```

### Step 3: Run the Application

Run the main Python file:

```
python main.py
```

The application will display the main menu in the terminal.

## 7. Main Menu

---

The application provides the following options:

1. Expense Management
2. Budget Management
3. Analytics
4. Financial Goals
5. Income Management
6. Final Report
7. Exit

Select an option by entering its number.

## 8. Running Tests

---

The project uses Python's built-in `unittest` framework.

Run all tests using:

```
python -m unittest discover -s tests -v
```

The project contains **22 test cases** covering:

- Expense management
- Budget management
- Analytics
- Financial goals
- Income management
- Report generation

The current test result is:

```
Ran 22 tests
```

```
OK
```

## 9. Data Storage

---

The application stores data in JSON files inside the `data` folder.

### `expenses.json`

Stores expense information such as:

- Expense ID
- Amount
- Category
- Description
- Date

### `budget.json`

Stores:

- Monthly budget
- Category-wise budgets

### `goals.json`

Stores:

- Goal ID
- Goal name
- Target amount
- Saved amount

### `income.json`

Stores the user's income amount.

The JSON files allow the information to remain available after the program is closed.

## 10. Error Handling and Validation

---

The project includes basic input validation for important values.

Examples include:

- Preventing zero or negative amounts.
- Preventing empty categories.
- Checking whether an expense or goal exists before editing or deleting it.
- Displaying messages when requested records are not found.
- Handling empty lists when there are no expenses or goals.

The validation is implemented using simple Python functions and conditional statements.

# 11. Course Relevance

This project applies concepts from **CSE1021 - Introduction to Problem Solving and Programming**.

The project uses:

- Variables and data types
- Input and output
- Arithmetic operations
- Conditional statements
- `for` loops
- `while` loops
- Functions
- Lists
- Dictionaries
- Searching
- Counting
- Summation
- Maximum and minimum operations
- File handling
- Modular programming

The project applies these concepts to solve a practical student expense-management problem.

# 12. Testing

Testing is performed using Python's built-in `unittest` framework.

The test suite covers the major modules of the application:

Test File	Area Tested
<code>test_expenses.py</code>	Expense management
<code>test_budget.py</code>	Budget management
<code>test_analytics.py</code>	Spending analytics
<code>test_goals.py</code>	Financial goals
<code>test_income.py</code>	Income management
<code>test_report.py</code>	Final report

A total of **22 tests** are included, and all 22 tests currently pass successfully.

## 13. Project Documentation

---

The project includes the following documentation:

- `README.md` - Project overview, setup, usage, testing, and technical information.
- `statement.md` - Problem statement, scope, objectives, requirements, modules, technology, storage, and course relevance.
- **Project Report** - Detailed documentation of the project including architecture, diagrams, implementation, testing, challenges, learnings, and future enhancements.

## 14. Future Enhancements

---

Possible future improvements include:

- Graphical user interface.
- Monthly spending charts.
- Exporting reports to PDF or Excel.
- Recurring expense support.
- Multiple user profiles.
- More advanced financial analysis.
- Automatic monthly summaries.

## 15. Author

---

**Shabd Mathur**

**Registration Number:** 26BAI10433

**VIT Bhopal University**

**B.Tech Computer Science and Engineering  
(Artificial Intelligence and Machine Learning)**